Отчёт по лабораторной работе №2

Первоначальная настройка git

true

2026-03-04

Отчёт по лабораторной работе №2

Первоначальная настройка git

Студент: Санджар Ниязов
Email: niazovsanzar08@gmail.com
Дата: 4 марта 2026 г.

Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

Ход работы

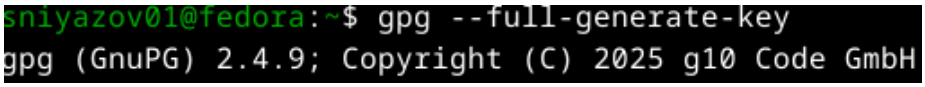
…

1. Проверка и настройка Git

Были проверены и настроены основные параметры Git (рис. \ref{fig:git-config):

2. Создание PGP-ключа

Создан PGP-ключ для подписи коммитов (рис. \ref{fig:gpg-create, \ref{fig:gpg-info, \ref{fig:gpg-list):



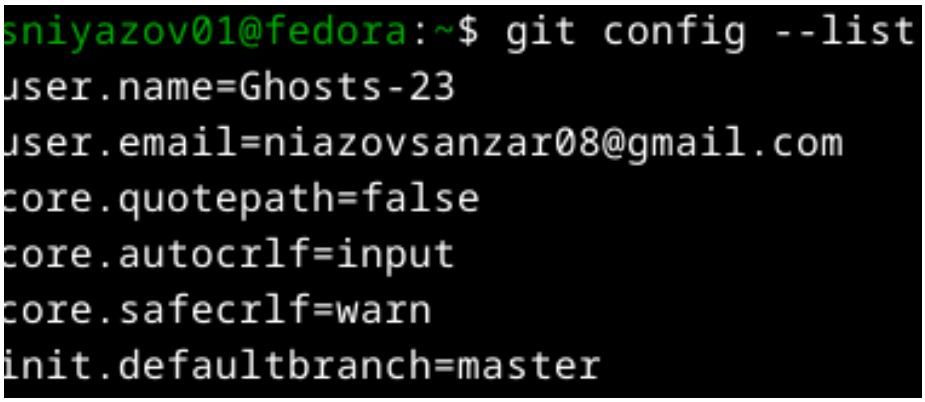
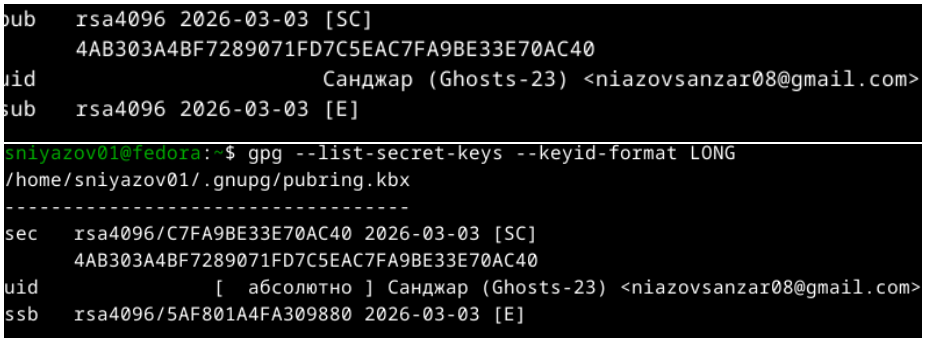
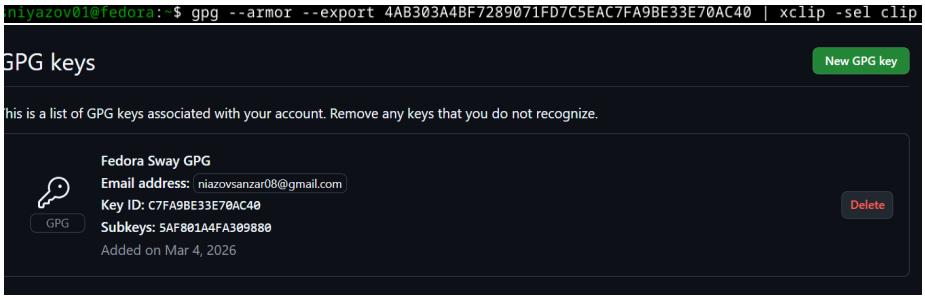


Figure 1: Настройки Git



3. Экспорт и добавление ключа на GitHub

Публичный ключ был экспортирован (рис. \ref{fig:gpg-export) и добавлен в настройках GitHub (рис. \ref{fig:github-gpg):



4. Настройка автоматической подписи коммитов

Настроена автоматическая подпись всех коммитов (рис. \ref{fig:git-signing):

```
sniyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git conf
ig --global --list | grep -E 'signingkey|gpgsign|gpg.program'
user.signingkey=4AB303A4BF7289071FD7C5EAC7FA9BE33E70AC40
commit.gpgsign=true
gpg.program=/usr/bin/gpg2
```

Figure 2: Настройка подписей

5. Создание подписанного коммита

Был создан тестовый файл (рис. \ref{fig:file-create), добавлен в git (рис. \ref{fig:git-add), выполнен подписанный коммит (рис. \ref{fig:git-commit) и отправлен на GitHub (рис. \ref{fig:git-push):

```
iyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ echo "# Лабораторная работа
" > lab02.md
iyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ cat lab02.md
Лабораторная работа №2
```

```
sniyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git add lab02.md
sniyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git status
Текущая ветка: master
Ваша ветка опережает «origin/master» на 1 коммит.
  (используйте «git push», чтобы опубликовать ваши локальные коммиты)

Изменения, которые будут включены в коммит:
  (используйте «git restore --staged <файл>...», чтобы убрать из индекса)
        новый файл:    lab02.md
```

```
niyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -S -m "feat(lab02
): add lab02 markdown file"
[master b20aae1] feat(lab02): add lab02 markdown file
 1 file changed, 1 insertion(+)
 create mode 100644 lab02.md
```

```
niyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git push
еречисление объектов: 21, готово.
одсчет объектов: 100% (21/21), готово.
ри сжатии изменений используется до 12 потоков
жатие объектов: 100% (14/14), готово.
апись объектов: 100% (16/16), 132.13 KiB | 1.06 MiB/s, готово.
otal 16 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
emote: Resolving deltas: 100% (2/2), completed with 2 local objects.
o github.com:Ghosts-23/study_2025-2026_os-intro.git
   5513fd1..b20aae1  master -> master
```

6. Проверка подписи на GitHub

На GitHub коммиты отображаются с пометкой Verified (рис. \ref{fig:github-verified):

7. Настройка GitHub CLI

Выполнена авторизация в GitHub CLI (рис. \ref{fig:gh-start, \ref{fig:gh-finish):

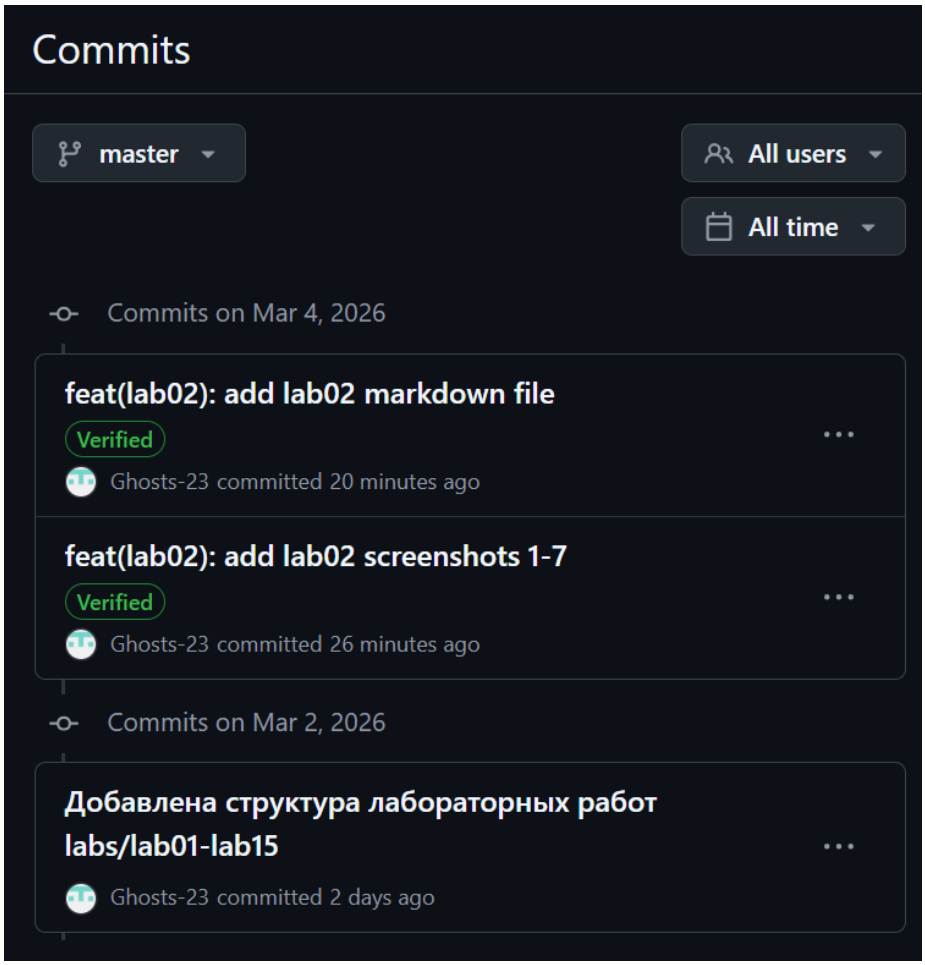


Figure 3: Verified коммиты

```
sniyazov01@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ cd ~/work/study/2025-
2026/Операционные\ системы/os-intro
gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/sniyazov01/.ssh/id_ed25519.pub
? Title for your SSH key: sniyazov01-fedora
? How would you like to authenticate GitHub CLI? Login with a web browser
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
! Authentication credentials saved in plain text
✓ SSH key already existed on your GitHub account: /home/sniyazov01/.ssh/id_ed25519.pub
✓ Logged in as Ghosts-23
```

Вывод

В ходе лабораторной работы были изучены средства контроля версий Git, создан PGP-ключ для подписи коммитов, настроена автоматическая подпись, выполнена интеграция с GitHub и GitHub CLI. Все коммиты успешно верифицируются на GitHub.

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (VCS) — это программные инструменты, которые помогают отслеживать изменения в файлах, координировать работу над проектами и хранить историю изменений. Они предназначены для: - Сохранения истории изменений файлов - Возможности отката к любой предыдущей версии - Совместной работы нескольких разработчиков - Разрешения конфликтов при одновременном редактировании - Ведения документации изменений (кто, когда и что изменил)

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- **Хранилище (repository)** — база данных, где хранятся все файлы и история их изменений. Может быть локальным или удалённым.
- **Commit (фиксация)** — сохранение изменений в хранилище с описанием того, что было сделано. Каждый коммит имеет уникальный идентификатор (хэш).
- **История (history)** — последовательность коммитов, отражающая эволюцию проекта. Позволяет просматривать изменения во времени.
- **Рабочая копия (working copy)** — текущая версия файлов проекта на локальном компьютере, с которой работает пользователь.

Отношения: Рабочая копия создаётся из хранилища. После внесения изменений пользователь делает коммит, который добавляется в историю хранилища.

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные VCS: - Единый центральный сервер с хранилищем - Клиенты получают рабочие копии - Для работы нужно подключение к серверу - Примеры: CVS, Subversion (SVN)

Децентрализованные (распределённые) VCS: - Каждый клиент имеет полную копию хранилища (включая историю) - Можно работать автономно - Примеры: Git, Mercurial, Bazaar

Главное отличие: В централизованных VCS история хранится только на сервере, в распределённых — у каждого разработчика есть полная копия истории.

4. Опишите действия с VCS при единоличной работе с хранилищем.

1. **Инициализация:** `git init` — создание локального репозитория
2. **Добавление файлов:** `git add file.txt` — добавление файлов в индекс
3. **Фиксация изменений:** `git commit -m "описание"` — создание коммита
4. **Просмотр состояния:** `git status` — проверка изменённых файлов
5. **Просмотр истории:** `git log` — просмотр истории коммитов
6. **Создание веток:** `git branch new-feature` — создание новой ветки
7. **Слияние веток:** `git merge new-feature` — объединение изменений

5. Опишите порядок работы с общим хранилищем VCS.

1. **Клонирование:** `git clone <url>` — получение удалённого репозитория
2. **Получение обновлений:** `git pull` — загрузка изменений из удалённого репозитория
3. **Создание ветки:** `git checkout -b feature-branch` — создание ветки для новой функциональности
4. **Внесение изменений:** редактирование файлов
5. **Коммит:** `git commit -am "описание изменений"` — фиксация изменений
6. **Отправка:** `git push` — отправка изменений в удалённый репозиторий
7. **Создание Pull Request** — запрос на слияние изменений (в GitHub/GitLab)

6. Каковы основные задачи, решаемые инструментальным средством git?

- Отслеживание истории изменений файлов
- Совместная работа нескольких разработчиков

- Управление ветками и слияниями
- Возможность отката к любой версии
- Резервное копирование кода
- Ведение документации изменений
- Разрешение конфликтов при параллельной работе

7. Назовите и дайте краткую характеристику командам git.

Команда	Назначение
`git init`	Создание нового репозитория
`git clone`	Копирование удалённого репозитория
`git add`	Добавление файлов в индекс
`git commit`	Фиксация изменений
`git status`	Просмотр состояния
`git log`	Просмотр истории
`git diff`	Просмотр изменений
`git branch`	Управление ветками
`git checkout`	Переключение между ветками
`git merge`	Слияние веток
`git pull`	Получение изменений из удалённого репозитория
`git push`	Отправка изменений в удалённый репозиторий
`git remote`	Управление удалёнными репозиториями

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

Локальный репозиторий: “‘bash mkdir project cd project git init echo “# My Project” > README.md git add README.md git commit -m “Initial commit” git branch feature git checkout feature echo “New feature” > feature.txt git add feature.txt git commit -m “Add new feature” git checkout master git merge feature

Удалённый репозиторий: git clone https://github.com/user/project.git cd project git checkout -b new-feature echo “Update” » file.txt git add file.txt git commit -m “Add new feature” git push origin new-feature

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветви (branches) — это отдельные линии разработки, позволяющие работать над разными функциями независимо друг от друга.

Зачем нужны:

- Разработка новых функций без влияния на основную ветку
- Эксперименты с кодом
- Исправление ошибок в отдельных версиях

- Параллельная работа нескольких разработчиков
- Организация процесса разработки (main, develop, feature branches)

10. Как и зачем можно игнорировать некоторые файлы при commit?

Для игнорирования файлов используется файл **.gitignore**.

Зачем игнорировать:

- Временные файлы (*.tmp,* .log)
- Скомпилированные файлы (*.o,* .class)
- Конфиденциальные данные (пароли, ключи)
- Файлы среды разработки (.idea/, .vscode/)
- Папки с зависимостями (node_modules/, vendor/)

Как игнорировать: *Создать файл .gitignore* echo “*.log” » .gitignore echo “node_modules/” » .gitignore echo “config/local.php” » .gitignore **Добавить .gitignore в репозиторий** git add .gitignore git commit -m “Add .gitignore”

Список литературы